WEEK 5

Payment Transaction Management System

Definition: A Payment Transaction Management System is a database application that stores and manages customer payment records. It tracks whether payments succeed or fail and produces reports about payment methods, revenue, and customer payment history.

1. Scenario

Customers pay for products or services through methods such as UPI, cash, net banking, and debit or credit cards. The system records each payment's amount, date, mode, and status. Staff can use these records to track transactions, calculate successful revenue, and understand customers' payment habits.

2. Tasks to Be Done

Task	Definition
Create a Payment table	Create a database table that stores one record for each payment transaction.
Store payment details	Save the payment ID, customer ID, amount, payment mode, status, and date.
Manage successful and failed transactions	Record the outcome of each attempt so completed payments can be separated from failed payments.
Analyze payment methods	Count and compare transactions made through each payment mode.
Report 1 — Payment mode analysis	Show how many transactions were made using each payment method.
Report 2 — Revenue analysis	Calculate total successful revenue and compare revenue across payment modes.
Report 3 — Customer payment history	Display a customer's past payment records, including amount, date, mode, and status.

3. Primary Entity: Payment

Definition: An entity is a real-world object or concept about which a database stores information. Payment is the primary entity here; each row in the Payment table represents one payment transaction.

Attribute	Definition
PAYMENT_ID	Unique identifier for each payment; used as the primary key.
CUSTOMER_ID	Identifier of the customer who made the payment.
TRANSACTION_AMOUNT	The amount of money paid or attempted.
PAYMENT_MODE	The method used to pay, such as UPI, Cash, Net Banking, or Card.
PAYMENT_STATUS	The result of the transaction, such as Success or Failed.
PAYMENT_DATE	The date on which the payment transaction occurred.

4. SQL Table Creation

Definition: CREATE TABLE defines the structure of a database table, including its columns, data types, and constraints.

```
CREATE TABLE Payment (
PAYMENT_ID NUMBER PRIMARY KEY,
CUSTOMER_ID NUMBER NOT NULL,
TRANSACTION_AMOUNT NUMBER(10,2) NOT NULL,
PAYMENT_MODE VARCHAR2(30) NOT NULL,
PAYMENT_STATUS VARCHAR2(20) NOT NULL,
PAYMENT_DATE DATE
);
```

5. SQL Queries with Definitions

Query 1 — Count UPI transactions

Definition: COUNT(*) counts all rows that match the condition. This query counts payments made through UPI.

```
SELECT COUNT(*) AS UPI_TRANSACTIONS
FROM Payment
WHERE PAYMENT_MODE = 'UPI';
```

Query 2 — Count card payments

Definition: This query counts payment records where the payment mode is Card.

```
SELECT COUNT(*) AS CARD_PAYMENTS
FROM Payment
WHERE PAYMENT_MODE = 'Card';
```

Report 1 — Payment mode analysis

Definition: GROUP BY collects records with the same payment mode. COUNT(*) counts transactions in each group, and ORDER BY sorts the largest count first.

```
SELECT PAYMENT_MODE, COUNT(*) AS TRANSACTION_COUNT
FROM Payment
GROUP BY PAYMENT_MODE
ORDER BY TRANSACTION_COUNT DESC;
```

Query 4 — Total successful revenue

Definition: SUM adds the transaction amounts. The WHERE condition includes only successful transactions, excluding failed attempts from revenue.

```
SELECT SUM(TRANSACTION_AMOUNT) AS TOTAL_REVENUE
FROM Payment
WHERE PAYMENT_STATUS = 'Success';
```

Query 5 — Most-used payment mode

Definition: This query returns the payment mode or modes with the highest transaction count. It uses a grouped count and compares it with the maximum count.

```
SELECT PAYMENT_MODE, COUNT(*) AS TRANSACTION_COUNT
FROM Payment
GROUP BY PAYMENT_MODE
HAVING COUNT(*) = (
SELECT MAX(TRANSACTION_COUNT)
FROM (
SELECT COUNT(*) AS TRANSACTION_COUNT
FROM Payment
GROUP BY PAYMENT_MODE
)
);
```

Report 2 — Revenue analysis by payment mode

Definition: This query adds successful transaction amounts separately for each payment mode and lists the highest revenue first.

```
SELECT PAYMENT_MODE,
SUM(TRANSACTION_AMOUNT) AS REVENUE
FROM Payment
WHERE PAYMENT_STATUS = 'Success'
GROUP BY PAYMENT_MODE
ORDER BY REVENUE DESC;
```

Query 7 — Average successful transaction

Definition: AVG calculates the mean transaction amount, considering only successful payments.

```
SELECT AVG(TRANSACTION_AMOUNT) AS AVERAGE_TRANSACTION_AMOUNT
FROM Payment
WHERE PAYMENT_STATUS = 'Success';
```

Report 3 — Customer payment history

Definition: This report lists payment records for all customers, grouped by customer and ordered from the most recent payment to the oldest. Add WHERE CUSTOMER_ID = 101 to view one customer's history.

```
SELECT PAYMENT_ID, CUSTOMER_ID,
TRANSACTION_AMOUNT, PAYMENT_MODE,
PAYMENT_STATUS, PAYMENT_DATE
FROM Payment
ORDER BY CUSTOMER_ID, PAYMENT_DATE DESC;
```

Successful and failed transaction summary

Definition: Conditional aggregation counts successful and failed payments separately in one result.

```
SELECT
SUM(CASE WHEN PAYMENT_STATUS = 'Success' THEN 1 ELSE 0 END) AS SUCCESS_COUNT,
SUM(CASE WHEN PAYMENT_STATUS = 'Failed' THEN 1 ELSE 0 END) AS FAILED_COUNT
FROM Payment;
```

6. Outputs Provided

The screenshots below show the SQL query outputs supplied for this activity.

```
SQL> SELECT COUNT(*) AS UPI_TRANSACTIONS FROM Payment WHERE PAYMENT_MODE='UPI';

UPI_TRANSACTIONS
----------------
               2
```

Output 1: UPI transaction count = 2

```
SQL> SELECT COUNT(*) AS CARD_PAYMENTS FROM Payment WHERE PAYMENT_MODE='Card';

CARD_PAYMENTS
-------------
            1
```

Output 2: Card payment count = 1

```
SQL> SELECT PAYMENT_MODE,COUNT(*) AS TRANSACTION_COUNT FROM Payment GROUP BY PAYMENT_MODE ORDER BY TRANSACTION_COUNT DESC;

PAYMENT_MODE         TRANSACTION_COUNT
-------------------- -----------------
UPI                                  2
Cash                                 1
Net Banking                          1
Card                                 1
```

Output 3: Payment mode counts — UPI: 2, Cash: 1, Net Banking: 1, Card: 1

```
SQL> SELECT SUM(TRANSACTION_AMOUNT) AS TOTAL_REVENUE FROM Payment WHERE PAYMENT_STATUS='Success';

TOTAL_REVENUE
-------------
         7200
```

Output 4: Total successful revenue = 7200

```
SQL> SELECT PAYMENT_MODE,COUNT(*) AS TRANSACTION_COUNT FROM Payment GROUP BY PAYMENT_MODE HAVING COUNT(*)=(SELECT MAX(COUNT(*)) FROM Payment GROUP B
Y PAYMENT_MODE);

PAYMENT_MODE         TRANSACTION_COUNT
-------------------- -----------------
UPI                                  2
```

Output 5: Most-used payment mode — UPI with 2 transactions

```
SQL> SELECT PAYMENT_MODE,SUM(TRANSACTION_AMOUNT) AS REVENUE FROM Payment WHERE PAYMENT_STATUS='Success' GROUP BY PAYMENT_MODE ORDER BY REVENUE DESC;

PAYMENT_MODE           REVENUE
-------------------- ----------
Net Banking                3500
UPI                        2500
Cash                       1200
```

Output 6: Successful revenue by mode — Net Banking: 3500, UPI: 2500, Cash: 1200

```
SQL> SELECT AVG(TRANSACTION_AMOUNT) AS AVERAGE_TRANSACTION_AMOUNT FROM Payment WHERE PAYMENT_STATUS='Success';

AVERAGE_TRANSACTION_AMOUNT
--------------------------
                      2400
```

Output 7: Average successful transaction amount = 2400

7. Report Analysis

Payment mode analysis: There are 5 transactions in total. UPI is the most frequently used method with 2 transactions. Cash, Net Banking, and Card each have 1 transaction.

Revenue analysis: The supplied output shows total successful revenue of 7200. Net Banking contributes 3500, UPI contributes 2500, and Cash contributes 1200. These reported mode totals add up to 7200.

Average transaction: The average amount of a successful transaction is 2400 according to the supplied output.

Customer payment history: The provided screenshots do not include the output of a customer-history query. The SQL query above can generate this report from the Payment table.

8. Conclusion

The Payment Transaction Management System helps maintain payment records, distinguish successful payments from failed attempts, and analyze payment preferences and revenue. SQL functions such as COUNT(), SUM(), and AVG(), along with GROUP BY, HAVING, and ORDER BY, support the three required reports and help administrators make informed decisions.